

ASSIGNMENT_2.5

2303A510H5

Batch_30

M.Parimala

Task 1: Refactoring Odd/Even Logic (List Version)

Prompt:

Write a program to calculate the sum of odd and even numbers in a list, then refactor it using AI.

Code:

```
def sum_odd_even(numbers):  
    odd_numbers = [num for num in numbers if num % 2 != 0]  
    even_numbers = [num for num in numbers if num % 2 == 0]  
    return sum(odd_numbers), sum(even_numbers)  
  
# Example usage:  
my_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
odd, even = sum_odd_even(my_list)  
print(f"Sum of odd numbers: {odd}")  
print(f"Sum of even numbers: {even}")  
  
... Sum of odd numbers: 25  
    Sum of even numbers: 30
```

Justification:

In this task, I was given a scenario where I had to improve an old or legacy Python program. The program was used to calculate the sum of odd and even numbers from a list. Initially, the program used a for loop with index values and separate variables to store the sums of odd and even numbers.

Although the program worked correctly, the code was long and not very clean. With the help of AI tools like Gemini and Cursor AI, I refactored the code to make it shorter and more readable. The improved version used Python's built-in `sum()` function and conditional expressions to calculate the sums directly.

This refactored code is easier to understand and maintain. It also follows better coding practices. From this task, I understood how AI tools can help in improving existing code and making it more efficient.

Task 2: Area Calculation Explanation

Prompt:

Ask Gemini to explain a function that calculates the area of different Shapes

Code:

```
def calculate_area(shape_type, **kwargs):
    if shape_type == 'circle':
        radius = kwargs.get('radius')
        if radius is not None: return 3.14159 * radius ** 2
        else: return "Error: Radius is required for a circle."
    elif shape_type == 'rectangle':
        length = kwargs.get('length')
        width = kwargs.get('width')
        if length is not None and width is not None: return length * width
        else: return "Error: Length and width are required for a rectangle."
    elif shape_type == 'triangle':
        base = kwargs.get('base')
        height = kwargs.get('height')
        if base is not None and height is not None: return 0.5 * base * height
        else: return "Error: Base and height are required for a triangle."
    else:
        return "Error: Unknown shape type."

# Example usage:
print(calculate_area('circle', radius=5,)) # Output: 78.53975
print(calculate_area('rectangle', length=4, width=6)) # Output: 24
print(calculate_area('triangle', base=3, height=8)) # Output: 12.0
```

Output:

```
... circle area: 78.53975  
    rectangle area: 24  
    triangle area: 12.0
```

justification:

The second task focused on explaining a Python function that calculates the area of different shapes. The scenario was that a junior developer is joining the team and needs to understand how the function works.

I wrote a function that calculates the area of a circle, rectangle, and triangle using conditional statements. I then used Google Gemini to explain the code. Gemini explained the purpose of each parameter, how the if and elif conditions work, and how the correct formula is selected based on the shape.

The explanation given by Gemini was simple and easy to understand. This made it helpful for beginners. From this task, I learned that Gemini is very useful for learning and teaching purposes because it explains code logic clearly.

Task 3: Prompt Sensitivity Experiment

Prompt:

Use Cursor AI with different prompts for the same problem and observe code changes

Code:

```
C: > Users > parim > Desktop > Untitled-1.py > ...
1  def sum_odd_even(numbers):
2      odd_sum = sum(num for num in numbers if num % 2 != 0)
3      even_sum = sum(num for num in numbers if num % 2 == 0)
4      return odd_sum, even_sum
5
6  # Example usage
7  numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
8  odd_sum, even_sum = sum_odd_even(numbers)
9  print(f"Sum of odd numbers: {odd_sum}")
10 print(f"Sum of even numbers: {even_sum}")
11
```

Output:

```
PS C:\Users\parim\Desktop> & 'c:\Users\parim\AppData\Local\Programs\Python\Python312\python.exe'
s\parim\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '6
-' 'c:\Users\parim\Desktop\Untitled-1.py'
Sum of odd numbers: 25
Sum of even numbers: 30
PS C:\Users\parim\Desktop>
```

Justification:

In this task, I tested how Cursor AI reacts to different prompts for the same programming problem. The problem was to write a function to check whether a number is prime. I gave Cursor AI different prompts such as a basic prompt, an optimized prompt, and a prompt asking for explanation.

Each prompt resulted in different versions of the code. When I asked for an optimized solution, the AI generated a faster and more efficient algorithm. When I asked for explanation, the AI added comments and made the code easier to understand.

This experiment helped me understand that AI tools are very sensitive to the way prompts are written. Clear and detailed prompts give better results. This task showed me the importance of writing good prompts while using AI tools.

Task 4: Tool Comparison Reflection

Prompt:

Based on your work in this topic, compare Gemini, Copilot, and Cursor AI for usability and code quality.

Code:

In the final task, I compared Google Gemini, GitHub Copilot, and Cursor AI based on my experience in this lab. Google Gemini is very good at explaining code and concepts, which makes it suitable for students and beginners. GitHub Copilot is useful for fast coding and real-time suggestions but does not explain the logic much.

Cursor AI combines both features. It helps in writing code, improving existing code, and explaining logic when asked. Because of this, I found Cursor AI to be the most useful tool for learning as well as development.

Based on my experience with Gemini, GitHub Copilot, and Cursor AI, all three are effective AI tools for programming, but they vary in terms of ease of use and the quality of code they generate.

Gemini is particularly strong in explaining concepts and providing guidance. It offers simple, well-organized code along with clear, beginner-friendly explanations, which makes it very suitable for learning and for helping new developers understand programming tasks.

GitHub Copilot is mainly focused on real-time development support. Since it works directly inside the code editor, it provides quick, context-based code suggestions. The code it generates is usually practical and reliable, making it a powerful tool for improving coding speed and overall productivity.

Cursor AI is especially useful for experimenting with prompts and improving existing code. It handles detailed instructions well, generates multiple versions of solutions, and is effective for refactoring, optimization, and working with legacy code.

In conclusion, Gemini is best for learning and concept clarification, Copilot excels in live coding assistance, and Cursor AI is most suitable for refining code and prompt-driven development.